

VIBE CODING FOR PROBLEM SOLVERS

O'Reilly Live Training · Instructor: Lucas Soares

7 Core Skills · 12 Patterns · Tool Landscape · Security Guidelines · 2025

WHAT IS VIBE CODING?

Vibe Coding

Building software with LLMs **without reviewing generated code**. Coined by Andrej Karpathy (Feb 2025). "Fully giving in to the vibes, embracing exponentials, and forgetting that the code even exists."

Use for: prototypes, experiments, throwaway projects, learning, MVPs, hackathons

AI-Assisted Programming

Using AI as a "typing assistant" **with human code review and oversight**. Simon Willison's Golden Rule: *"You won't commit code you couldn't explain to someone else."*

Use for: production systems, security-critical apps, shared/maintained codebases

Vibe Coding vs AI-Assisted Programming

Aspect	Vibe Coding	AI-Assisted Programming
Code Review	None — accept without inspection	Thorough human review required
Use Case	Prototypes, experiments, throwaway	Production, critical, maintained code
Speed	Maximum velocity	Balanced speed + quality
Risk	High (acceptable for low stakes)	Low (quality & security critical)
Skill Focus	Prompt engineering, system design	Programming fundamentals + AI collab

THE 7 CORE SKILLS

1. Prompting

Clear, specific instructions with examples.

Formula: Context + Task + Examples + Constraints + Format

6 Sub-Skills:

- **Clear & Direct** — specific, unambiguous requests (always use)
- **Decomposition** — break complex tasks into steps
- **Examples** — show desired input/output pairs
- **Role Assignment** — define AI expertise/persona
- **Time to Think** — request chain-of-thought reasoning
- **Constraints** — set tech stack, code size limits

2. Context Management

Strategic information loading.

Formula: Right context + Right amount + Right time = Better outputs

4 Strategies:

- **Write** — save to CLAUDE.md / external memory (reuse across sessions)
- **Select** — RAG, @-mentions, smart file selection (large codebases)
- **Compress** — summaries, handoffs, sliding windows (long convos)
- **Isolate** — split context across multiple agents

3. Capability Assignment

Know what to delegate vs. keep human.

Delegate to AI:

- Boilerplate, scaffolding, docs, tests
- Standard pattern implementations
- Data transformation & batch processing

Keep Human Control:

- Architecture decisions, system design
- Security-critical code, final QA
- Performance optimization

4. Vibe Checking

Lightweight verification without deep code review.

60-Second Routine:

- **Run it** (5s) — does it execute without error?
- **Check counts** (10s) — right amount of output?
- **Spot check** (15s) — first, last, random item look right?
- **Boundary test** (30s) — edge cases work? (zero, null, empty)

If all pass — you're good for vibe coding purposes.

5. Strategic Cognitive Offloading

The **70/30 Rule** for AI collaboration:

70% AI Responsibility:

- Routine implementation
- Pattern following, code generation
- Boilerplate & documentation

30% Human Responsibility:

- Strategic decisions
- Quality control
- Creative problem-solving

6. Personal Benchmarking

Build custom evaluation frameworks for your AI workflow.

Track These Metrics:

- **Speed** — development velocity improvements
- **Quality** — bug rates, user satisfaction scores
- **Learning** — skill development curve over time
- **ROI** — cost vs. benefit analysis

Teams report 30–40% reduction in sprint completion times.

7. Agentic Task Orchestration

Coordinating multiple AI agents for complex projects.

Workflow Steps:

- **Task decomposition** — break into agent-specific work
- **Workflow management** — sequence + parallel processing
- **Quality gates** — checkpoints between agent handoffs
- **Integration testing** — ensure agent outputs work together

Iterative Refinement Cycle

1. Conceptualize → 2. AI Generate → 3. Functional Test →
4. Feedback & Refine → 5. Iterate

Before Prompting, Ask:

Is it specific enough? · Should I decompose? · Would an example help? · Need specialized expertise? · Ask for reasoning? · Set constraints?

VIBE CHECKING — 8 VERIFICATION TECHNIQUES

1. Spot Check

Check first, last, and a random middle item. Best for: data transformations, batch processing.

2. Count Validation

Verify input/output quantities match — or filter logic is correct. `assert len(input) == len(output)`. Best for: parsing, filtering.

3. Boundary Test

Test zero, negative, large numbers, empty input, None/null. Best for: functions, validators, calculators.

4. Smoke Test

`python script.py && echo "SUCCESS"` — Does it run at all? Best for: any generated code — always your first check.

5. Format Validation

Check structure with `json.loads()` or `jsonschema.validate()`. Best for: data transforms, API responses.

6. Diff Check

`git diff --stat` — compare before/after. `diff original.txt refactored.txt`. Best for: refactoring, migrations.

7. Console Log Test

Add strategic `print()` statements to trace execution flow at key points. Best for: debugging, understanding new code.

8. Type Check

`print(type(result))` — verify types are what you expect. Run `mypy script.py`. Best for: type-sensitive operations.

CONTEXT MANAGEMENT DECISION TREE

1

Reusable across sessions?

Write to `CLAUDE.md` or external docs

2

Codebase too large to load?

Select: @-mentions, RAG, targeted file selection

3

Conversation too long?

Compress: summarize, handoff to new session

4

Multiple specialists needed?

Isolate: split context across agents

12 PRACTICAL PATTERNS

1. Start with Prototypes

Fuzzy idea → working prototype → iterate. Beats analysis paralysis. When: new features, unclear requirements.

2. Vibe Checking Checklist

Green/red flag review before accepting. Run the 60-second routine on every AI-generated piece.

3. Provide Clear Specs

Detailed requirements before generation. Specificity directly correlates with output quality.

4. Restart When Stuck

Strategic context reset. Long sessions cause quality degradation — fresh context often wins.

5. Let It See

Share screenshots, error messages, logs. Bridges context gaps especially for UI and debugging.

6. Automate Steps

UV scripts for repeatable tasks: resize images, process CSV, parse meeting notes, scrape headlines.

7. Personal Benchmarks

Build your own eval criteria. Track velocity, quality, learning curves to prove ROI.

8. Store Format Prompts

Save effective prompts for reuse. Build a personal library of high-performing prompt templates.

9. Generate Synthetic Data

Create test datasets with AI. Faster than manual creation; great for edge case testing.

10. Voice Workflows

Dictate requirements, review output verbally. Faster for ideation and requirement capture.

11. Context-Based Coding

Load full codebase via gitingest, repomix, or rjina.ai before tackling complex tasks.

12. Raw → Structured

Transform notes/logs/natural language into structured data. Meeting notes → action items JSON.

TOOL LANDSCAPE (2025)

AI Coding Tools — Categories & Best Uses

Tool	Category	Best For	Price
Cursor	IDE Integration	Best overall; multi-file tasks, refactoring, serious projects	£16/mo
GitHub Copilot	IDE Integration	Traditional devs; fast autocomplete in existing IDE	£10/mo
Windsurf	IDE Integration	Spec-driven dev, teams wanting structured process	£15/mo
Claude Code	Terminal Agent	Complex agent workflows, MCP integration, full-stack	—
Aider	Terminal Agent	Git-aware terminal coding, open source	Free
Replit	Cloud / No-Code	Beginners, collaboration (75% of users never write code)	Free/£20
Lovable	Cloud / No-Code	Non-technical founders; full-stack apps from conversation	£25/mo
Bolt.new	Cloud / No-Code	Rapid prototypes, hackathons, instant preview	Free–£100
Claude Artifacts	Web Builder	Quick one-shot prototypes directly in browser chat	—
Vercel v0	Web Builder	UI/React component generation, design-to-code	Free tier

WHEN TO VIBE CODE VS. AVOID

✓ SAFE — Vibe Code Here

- Throwaway weekend projects
- Prototypes & proof-of-concepts
- Low-stakes experiments
- Personal scripts & automation tools
- Learning new frameworks/libraries
- MVP development & hackathons
- Internal tools (low stakes)
- Generating test data & mock APIs

✗ AVOID — Use AI-Assisted Instead

- Production user-facing systems
- Security-critical code (auth, payments)
- Regulated industries (healthcare, finance)
- Long-term maintained codebases
- Shared team codebases
- High-scale / real-time systems
- Code with legal/compliance obligations
- When downtime has significant business impact

Risk Assessment — Before Choosing Vibe Coding, Ask:

1. **Impact of failure** — what happens if code breaks? · 2. **Data sensitivity** — personal or financial info? · 3. **Maintenance** — long-term updates needed? · 4. **Team** — do others maintain it? · 5. **Compliance** — regulatory obligations? **High Risk = Avoid Vibe Coding**

SECURITY PITFALLS IN AI-GENERATED CODE

SQL Injection — 40% of AI queries vulnerable

```
# VULNERABLE
query = "SELECT * FROM users WHERE id = " + user_id

# SECURE — parameterized query
query = "SELECT * FROM users WHERE id = ?"
cursor.execute(query, [user_id])
```

Hardcoded Credentials

```
# VULNERABLE
api_key = "sk-1234567890abcdef"

# SECURE — environment variables
import os
api_key = os.environ.get('API_KEY')
```

XSS — Cross-Site Scripting

```
// VULNERABLE — renders raw HTML
element.innerHTML = user_input;

// SECURE — renders as text only
element.textContent = user_input;
```

Security Best Practices

- **OWASP LLM Top 10:** Prompt Injection, Sensitive Data Disclosure
- Automated scanning: SonarQube, Snyk, CodeQL in CI/CD
- Never accept AI-generated auth/payment code without review
- Security-first prompting: specify OWASP, bcrypt, rate limiting, HTTPS-only
- Dependency vulnerability scanning on every build
- Regular penetration testing for production systems

CONTEXT ENGINEERING TOOLS

gitingest.com

Convert GitHub repos into LLM-ready text context. Paste the repo URL to get a full digest.

repomix

Package a local repository into a single AI-readable file. Ideal for large codebases.

r.jina.ai

Extract clean web content for AI context. Prefix any URL: `r.jina.ai/https://...`

KEY TAKEAWAYS

- **Know your mode:** Vibe coding ≠ all AI coding. Decide consciously which approach fits your risk profile.
- **Prompt quality is everything:** Context + Task + Examples + Constraints + Format = dramatically better outputs.
- **Verify smartly, not exhaustively:** 60-second check — run it, count outputs, spot check, boundary test.
- **70/30 rule:** AI handles 70% (routine code), you handle 30% (strategy, quality, creative decisions).
- **Context is fuel:** Load the right context at the right time in the right amount — avoid overloading.
- **Security stays human:** Never accept AI-generated auth, payment, or security-critical code without review.
- **Restart beats debugging:** When sessions degrade in quality, a fresh context window often solves it faster.